

TP3 - Apprentissage automatique

Bastien Jallais

Automne 2025

Contents

1	Introduction	2
2	Montage de l'architecture et entraînement du modèle	2
2.1	Ensemble de données	2
2.2	Traitement de données	2
2.3	Paramètres et Hyperparamètres	3
2.4	Architecture de l'autoencodeur	4
2.5	Affichage des résultats d'entraînement	6
2.6	Justification du choix de l'architecture	7
3	Évaluation du Modèle	8
3.1	Affichage des résultats de l'évaluation	8
3.2	Analyse et discussion des résultats de l'évaluation du modèle	10
4	Conclusion	10

1 Introduction

Ce travail porte sur la conception puis l'évaluation d'un autoencodeur. L'objectif est d'apprendre une représentation (embedding) d'images à partir d'un ensemble de données composé de deux classes : requin et dauphin. On souhaite également assurer une reconstruction fidèle des images en entrée.

Dans un premier temps, l'autoencodeur est conçu et entraîné afin de reconstruire les images à partir de leur représentation latente. L'entraînement est évalué avec une fonction de perte sur l'ensemble d'entraînement et de validation.

Puis, dans un second temps, la partie encodeur du modèle est utilisée pour extraire les embeddings des images. Ces représentations sont ensuite utilisées pour une tâche de classification à l'aide d'un classifieur SVM. Pour finir, des visualisations et des métriques de performances sont utilisées afin d'analyser efficacement les résultats obtenus.

2 Montage de l'architecture et entraînement du modèle

2.1 Ensemble de données

Les données d'entraînement sont divisées en deux parties. On suit ici une séparation classique de 80% de données pour le training et 20% pour les données de validation. Ce découpage est effectué à l'aide d'une cellule en utilisant google colab. On découpe en 2 sous dossiers donnees/validation et donnees/entrainement, voici le résultat de la cellule.

```
1 Classes dtectes: ['requin', 'dauphin']
2 [requin] total=1800 -> moved_to_val=360 remaining_train=1440
3 [dauphin] total=1800 -> moved_to_val=360 remaining_train=1440
4
5 Total dplac vers validation: 720
6 Train dir: donnees/entrainement
7 Val dir: donnees/validation
```

```
1 training_ds_size = 2880 # total 2880 (1440 classe: requin et 1440 classe: dauphin)
2 validation_ds_size = 720 # total 720 (360 classe: requin et 360 classe: dauphin)
```

2.2 Traitement de données

Après analyse des données, on remarque que toutes les images ont une largeur ou une hauteur autour de 256 pixels. Les images ayant donc des ratios variables ont été redimensionnées à 128×128 pour l'entraînement. Une amélioration possible serait d'appliquer un redimensionnement conservant le ratio avec padding afin de limiter la distorsion. une augmentation à 160 pourrait être envisagée, mais cela serait au prix d'un temps d'entraînement beaucoup plus élevé. 128 permettra un entraînement rapide mais précis, tout en évitant le risque de surapprentissage. Aussi cette taille est donnée comme standard dans les exemples de Keras, preuve de son efficacité.

```
1 image_scale = 128
```

On modifie également les valeurs de image_channels et images_color_mode à respectivement 3 et "rgb" car les images qu'on traite ici sont en couleur et non en noir et blanc.

```
1 image_channels = 3
2 images_color_mode = "rgb"
```

Enfin une augmentation de données est effectuée uniquement sur l'ensemble d'entraînement. On effectue une rotation de $\pm 10^\circ$, translations $\pm 5\%$, zoom 10% et un flip horizontal. Cela permet d'améliorer la capacité de généralisation du modèle.

```

1 training_data_generator = ImageDataGenerator(
2     rescale=1./255,
3     rotation_range=10,
4     width_shift_range=0.05,
5     height_shift_range=0.05,
6     zoom_range=0.10,
7     horizontal_flip=True,
8     fill_mode='nearest'
9 )

```

2.3 Paramètres et Hyperparamètres

La taille du batch est de 32, cette valeur standard permet de stabiliser le gradient avec un bon compromis entre un gradient trop bruité et un gradient trop lissé. Aussi cette valeur donne les meilleurs résultats au niveau du temps d'entraînement lorsqu'elle est couplée avec le GPU de Colab. Pour finir, des tests avec une taille de 64 pour le batch ont montré une tendance à arrêter prématurément l'entraînement. C'est pourquoi une taille de 32 est finalement retenue.

```

1 fit_batch_size = 32 # le nombre d images entrainees ensemble: un batch

```

Le nombre d'époques retenu est de 60, les premiers essais montraient que l'accuracy continuait de s'améliorer jusqu'à 45-50 époques, au-delà, la courbe de validation se stabilise. Pour chercher la meilleure précision possible, on continue l'entraînement tout en rajoutant un *early stop*, comme présenté plus bas.

```

1 fit_epochs = 60 # Le nombre d'epoques

```

Un early stop a été mis en place avec une patience de 10 pour éviter que le modèle ne s'arrête trop tôt tout en optimisant la précision de celui-ci en lui laissant le temps de s'améliorer correctement. Cette early stop permet également d'éviter l'overfitting.

```

1 earlystop = EarlyStopping(
2     monitor='val_loss',
3     patience=10,
4     restore_best_weights=True
5 )

```

Enfin, l'optimisateur utilisé pour l'entraînement de l'autoencodeur est Adam. Celui-ci combine les avantages de la descente de gradient adaptative ainsi que le momentum. Un taux d'apprentissage de 0.001 est utilisé, c'est une valeur standard reconnue offrant un bon compromis en vitesse de convergence et stabilité de l'entraînement.

```

1 model.compile(loss='mse', optimizer=Adam(learning_rate=1e-3), metrics=['mse'])

```

2.4 Architecture de l'autoencodeur

L'architecture de l'autoencodeur se compose d'un encodeur ainsi qu'un décodeur symétrique à celui-ci. L'encodeur comprend trois blocs convolutionnels successifs. Chaque bloc est constitué de deux couches Conv2D avec un noyau de taille 3x3 et d'un padding de type same, suivies d'une Batch Normalization et d'une fonction d'activation ReLU. Chaque bloc se termine par une couche MaxPooling2D de taille 2x2. Un Dropout est appliqué après les deux premiers blocs de l'encodeur, avec des taux respectifs de 0.10 et 0.15. La sortie de l'encodeur correspond à un embedding de dimension $16 \times 16 \times 128$.

Le décodeur est construit de manière symétrique à l'encodeur. Il est composé de trois blocs comprenant chacun deux couches Conv2D (3×3), une Batch Normalization, une activation ReLU, suivies d'un UpSampling2D (2×2) permettant de restaurer progressivement la résolution spatiale. La couche de sortie est une Conv2D à 3 filtres, suivie d'une activation Sigmoid, produisant une image reconstruite de taille $128 \times 128 \times 3$.

Composant	Couches	Taille de sortie
Entrée	Image RGB	$128 \times 128 \times 3$
Encodeur – Bloc 1	Conv2D(32, 3×3) → BN → ReLU Conv2D(32, 3×3) → BN → ReLU MaxPooling2D(2×2) → Dropout(0.10)	$64 \times 64 \times 32$
Encodeur – Bloc 2	Conv2D(64, 3×3) → BN → ReLU Conv2D(64, 3×3) → BN → ReLU MaxPooling2D(2×2) → Dropout(0.15)	$32 \times 32 \times 64$
Encodeur – Bloc 3	Conv2D(128, 3×3) → BN → ReLU Conv2D(128, 3×3) → BN → ReLU MaxPooling2D(2×2)	$16 \times 16 \times 128$
Décodeur – Bloc 1	Conv2D(128, 3×3) → BN → ReLU Conv2D(128, 3×3) → BN → ReLU UpSampling2D(2×2)	$32 \times 32 \times 128$
Décodeur – Bloc 2	Conv2D(64, 3×3) → BN → ReLU Conv2D(64, 3×3) → BN → ReLU UpSampling2D(2×2)	$64 \times 64 \times 64$
Décodeur – Bloc 3	Conv2D(32, 3×3) → BN → ReLU Conv2D(32, 3×3) → BN → ReLU UpSampling2D(2×2)	$128 \times 128 \times 32$
Sortie	Conv2D(3, 3×3) → Sigmoid	$128 \times 128 \times 3$

Table 1: Architecture de l'autoencodeur convolutionnel utilisé. BN : Batch Normalization.

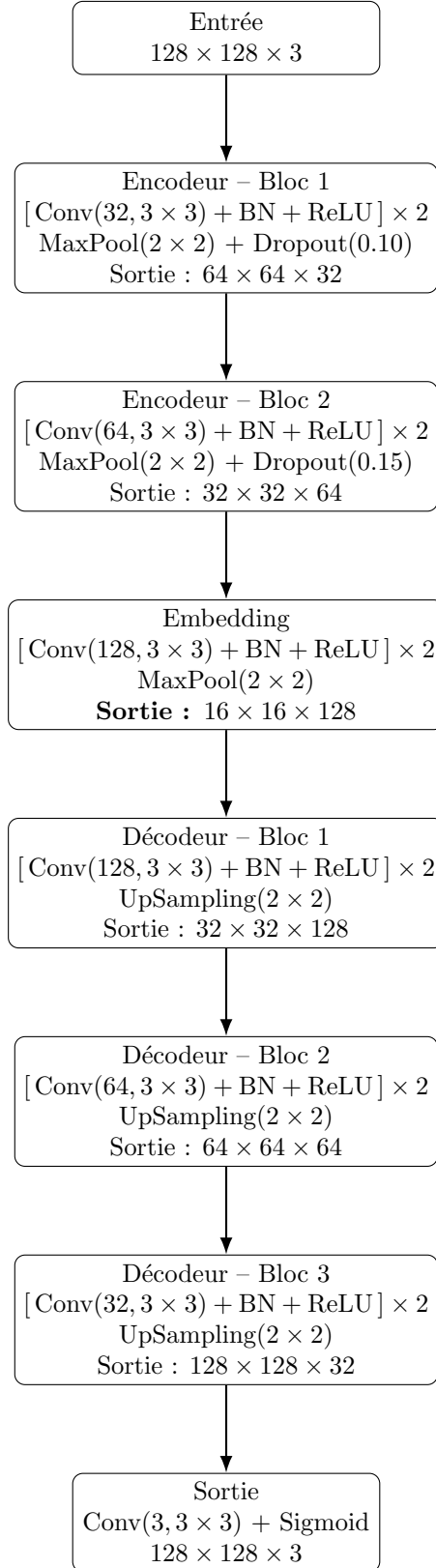


Figure 1: Schéma simplifié de l'architecture de l'autoencodeur convolutionnel (tailles des tenseurs et opérations principales).

2.5 Affichage des résultats d'entraînement

L'entraînement du modèle a duré 11.36 minutes. La courbe de perte par époque est présentée à la Figure 2. L'erreur minimale obtenue est de 0.002634 sur l'ensemble d'entraînement et de 0.003752 sur l'ensemble de validation, avec une meilleure performance atteinte à l'époque 44.

```
1 Temps total d execution (entrainement) : 11.36 minutes
2 Figure(800x500)
3 Figure sauvegardee : loss_curve.png
4 Min training loss: 0.002634
5 Min validation loss: 0.003752
6 Best epoch (min val_loss): 44
```

La Figure 2 montre une diminution rapide de la perte (MSE) au cours des premières époques, indiquant que l'autoencodeur apprend efficacement à reconstruire les images. Après environ 15–20 époques, la courbe de perte se stabilise progressivement, ce qui suggère une convergence du modèle.

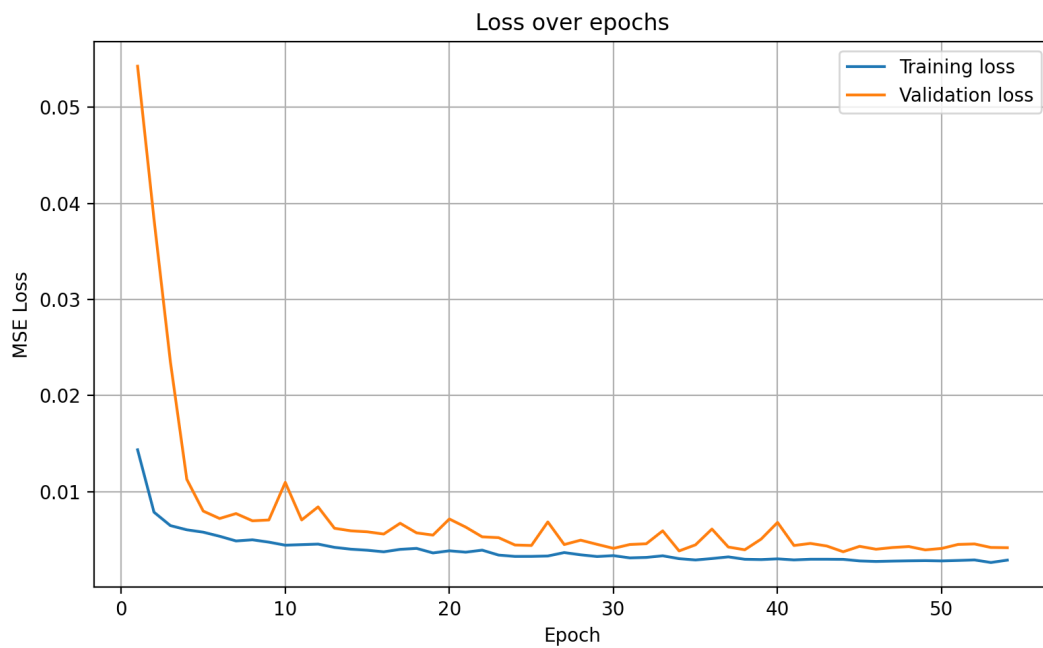


Figure 2: Évolution de la fonction de perte (MSE) au cours de l'entraînement.

2.6 Justification du choix de l'architecture

Le développement de l'autoencodeur a été réalisé de manière itérative, en partant d'une architecture simple composée de deux blocs dans l'encodeur et le décodeur. Cette première version permettait une reconstruction correcte des images, mais présentait une capacité limitée à généraliser. La perte minimale obtenue lors de la première version sur l'ensemble de validation était d'environ 0.005.

Dans le but d'améliorer la qualité et la stabilité de l'entraînement, un troisième bloc a été ajouté à l'encodeur et au décodeur. L'augmentation de la profondeur du réseau a permis de mieux extraire des caractéristiques, ce qui a conduit à une amélioration de la qualité de la reconstruction.

L'intégration de Batch Normalization après chaque couche de convolution permet de stabiliser l'apprentissage en réduisant les variations d'activation entre les différentes couches. L'utilisation de la fonction ReLU dans les couches internes facilite l'optimisation du réseau en limitant les effets de saturation des gradients. Pour finir, l'introduction d'un Dropout modéré dans l'encodeur (taux de 0.10 et 0.15) a permis de réduire le surapprentissage observé lors des premiers tests.

En complément, une augmentation de données a été appliquée à l'ensemble d'entraînement. Ce traitement permet d'exposer le modèle à des variations, ce qui renforce sa capacité de généralisation.

L'ensemble de ces changements combinés s'est traduit par une amélioration notable de la performance du modèle : la perte minimale sur l'ensemble de validation est passée d'environ 0.005 à 0.003, ce qui indique une meilleure capacité de généralisation. Enfin, l'utilisation de l'optimiseur Adam avec un taux d'apprentissage de 0.001, ainsi que le mécanisme d'early stopping ont permis de sélectionner automatiquement le meilleur modèle. La seule remarque qu'on peut faire entre la première version et la version finale, est que la durée d'entraînement du modèle est passée de 5 minutes à 11 minutes. Néanmoins cette augmentation reste anecdotique car ces valeurs restent dans le même ordre de grandeur et le but du TP est surtout de construire le modèle le plus performant.

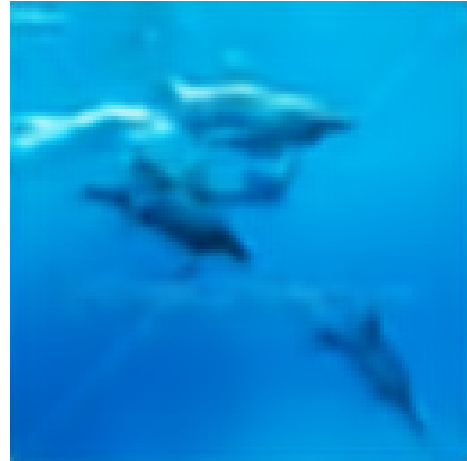
3 Évaluation du Modèle

3.1 Affichage des résultats de l'évaluation

Affiche d'une image de chaque classe sous sa forme originale et reconstruite



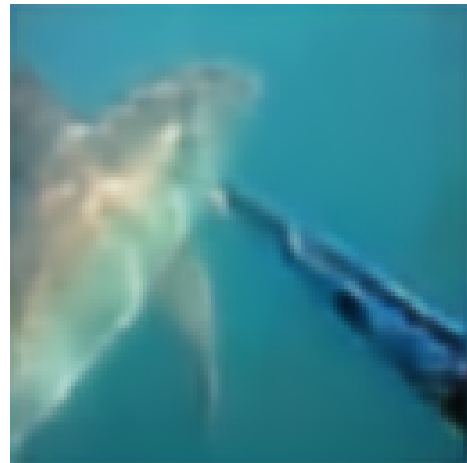
(a) Classe Dauphin – Originale



(b) Classe Dauphin – Reconstruite



(c) Classe Requin – Originale



(d) Classe Requin – Reconstruite

Figure 3: Exemples d'images originales et reconstruites par l'autoencodeur pour chaque classe du jeu de test.

Résultat de l'entraînement du SVM-Linéaire sur le embedding

La première étape consiste à faire une extraction de l'embedding avec l'encodeur sans oublier d'utiliser flatten afin d'obtenir un vecteur de représentation associé aux données de test. Voici ce qu'on obtient :

1	Shape embedding (avant flatten) : (600, 128, 128, 32)
2	Shape embedding_flat : (600, 524288)

Après aplatissement (flatten), chaque image est représentée par un vecteur de 524 288 caractéristiques, utilisé ensuite pour la normalisation, la classification SVM et la visualisation.

Les vecteurs d'embedding aplatis sont ensuite normalisés à l'aide d'un StandardScaler afin de centrer les données.

```
1 Shape embedding normalise : (600, 524288)
2 Moyenne (0) : -1.2603899e-08
3 cart-type (1) : 0.99963886
```

On entraîne ensuite un SVM directement sur les images originales, l'accuracy moyenne obtenue est de 0.5917. Elle sert de base pour comparer ensuite l'intérêt de l'embedding appris par l'autoencodeur.

```
1 {'svm__C': 0.001, 'svm__class_weight': None}
2 Meilleure accuracy CV (pixels): 0.5917
```

On applique ensuite un SVM Linéaire sur le flattened embedding normalisé. Étant donné la dimension très élevée de l'embedding flatten, une PCA est appliquée. On obtient une accuracy moyenne de $0,648 \pm 0,042$. Ce résultat est significativement supérieur à celui obtenu sur les pixels bruts, ce qui indique que l'autoencodeur a appris une représentation latente plus discriminante.

```
1 Accuracy SVM (embedding + PCA256) - CV 5-fold : 0.6483 0.0420
```

Pour finir on visualise l'embedding en deux dimension en affichant un scatter plot en utilisant TSNE-2D.

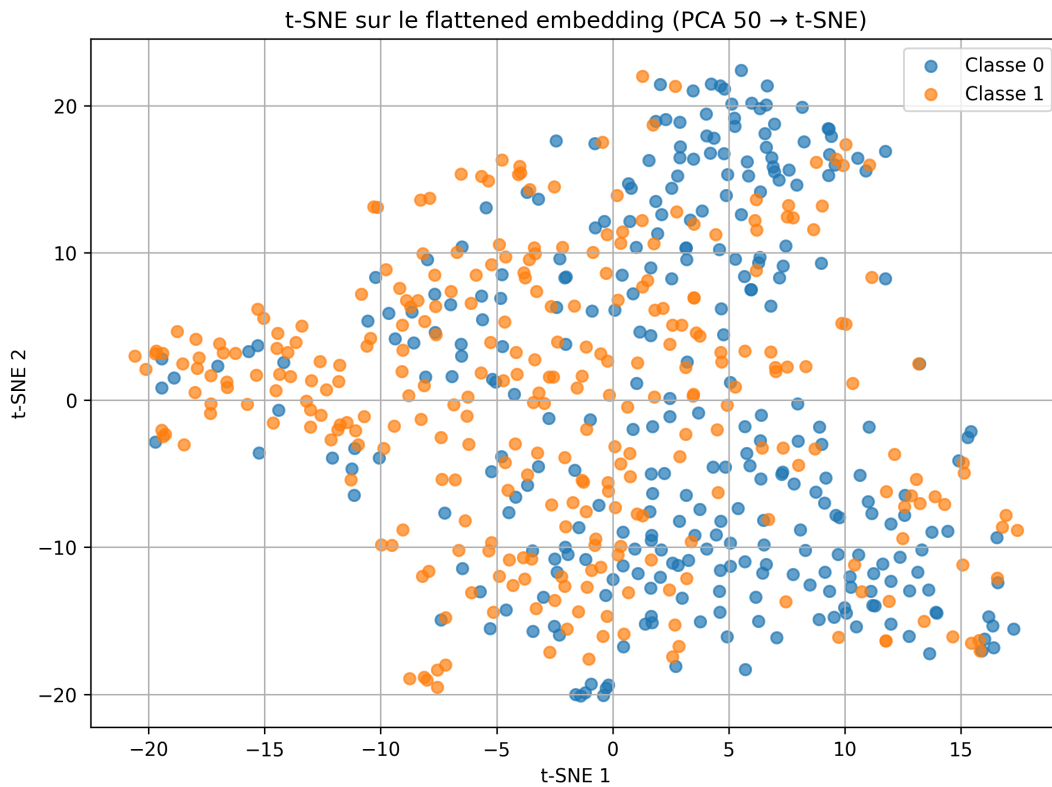


Figure 4: Scatter plot du embedding associé aux données de test.

3.2 Analyse et discussion des résultats de l'évaluation du modèle

Fidélité de la reconstruction

On remarque que l'autoencodeur parvient à préserver les structures globales des images originales pour les deux classes. Les contours principaux ainsi que la disposition des éléments sont globalement bien restitués après reconstruction.

Cependant, on observe une perte des détails, notamment au niveau des textures ou des contours précis. Malgré cela, la reconstruction reste suffisamment fidèle.

Séparation des classes dans le scatter plot

La visualisation de l'embedding en deux dimensions montre une séparation partielle mais néanmoins visible des deux classes. Bien que les points ne forment pas deux clusters totalement distincts, on peut remarquer une tendance claire.

Ce chevauchement peut s'expliquer notamment par la complexité visuelle des images marines, où certaines caractéristiques (formes, couleurs, arrière-plan) peuvent être partagées entre dauphins et requins.

Séparation des classes dans le scatter plot

L'entraînement du SVM linéaire directement sur les pixels bruts des images de test donne une accuracy moyenne de 0.5917. On obtient donc un résultat plutôt modeste, on peut expliquer cela par le fait que les pixels bruts contiennent beaucoup d'informations redondantes ou non pertinentes.

Après normalisation de l'embedding et réduction de la dimension par PCA, l'application du SVM linéaire atteint une accuracy moyenne de $0.648 \pm 0,042$, soit une amélioration significative par rapport aux pixels bruts. Ce gain de performance démontre que l'autoencodeur a appris une représentation plus compacte et plus discriminante que les données originales. L'embedding capture des caractéristiques de haut niveau pertinentes pour distinguer les deux espèces.

4 Conclusion

Le but du travail était de construire et d'entraîner un autoencodeur afin d'apprendre la représentation de deux espèces marines : dauphins et requins. L'objectif était dans un premier temps d'assurer une reconstruction fidèle des images d'entrée, et dans un deuxième temps d'analyser la qualité de l'embedding appris à l'aide d'un SVM linéaire.

Les résultats obtenus démontrent que l'autoencodeur parvient à reconstruire correctement les structures globales des images. Les formes générales et la disposition des éléments sont bien préservées, bien que certains détails, notamment les textures, soient atténués.

Malgré les nombreuses optimisations apportées à l'architecture — augmentation de la profondeur du réseau, utilisation de Batch Normalization, introduction de Dropout, data augmentation, early stopping et choix d'hyperparamètres adaptés — la séparation des classes à l'aide d'un SVM linéaire reste partielle. Bien que l'accuracy obtenue sur l'embedding soit supérieure à celle obtenue sur les pixels bruts, les résultats montrent qu'il reste difficile d'obtenir une frontière de décision clairement définie entre les deux classes.

Cette difficulté s'explique par le fait que les images marines présentent une forte variabilité intra-classe et une similarité visuelle importante entre dauphins et requins, notamment en termes de formes, de couleurs et de contextes visuels (fonds marins similaires, angles de vue variés, conditions d'éclairage). Ces éléments limitent la capacité d'un classifieur linéaire à séparer parfaitement les données, même lorsque l'embedding est de bonne qualité.

Plusieurs pistes d'amélioration peuvent être envisagées pour pallier ces limites. L'introduction d'une contrainte supervisée sur l'espace latent, par exemple via un apprentissage semi-supervisé ou l'ajout d'une tête de classification conjointe à l'autoencodeur, pourrait favoriser une meilleure séparation des classes. L'utilisation d'un classifieur non linéaire ou l'exploration d'architectures plus avancées, telles que des autoencodeurs variationnels ou des réseaux avec connexions résiduelles, pourraient également améliorer la structuration de l'espace latent.

En conclusion, ce travail met en évidence à la fois les forces et les limites des autoencodeurs convolutionnels pour l'apprentissage de représentations visuelles. Il démontre que, malgré une architecture optimisée et un entraînement soigné, la séparation des classes reste un défi lorsque l'apprentissage est non supervisé et que les données présentent un fort chevauchement visuel.